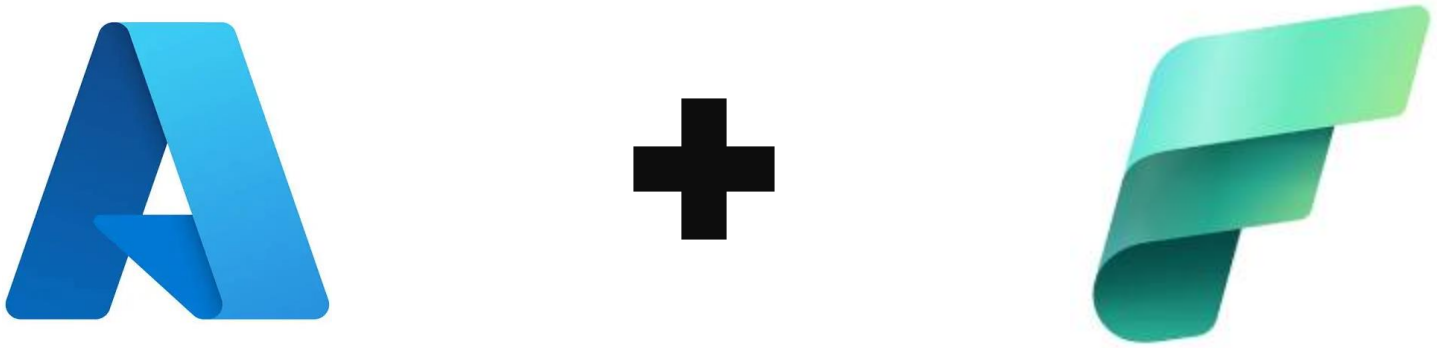
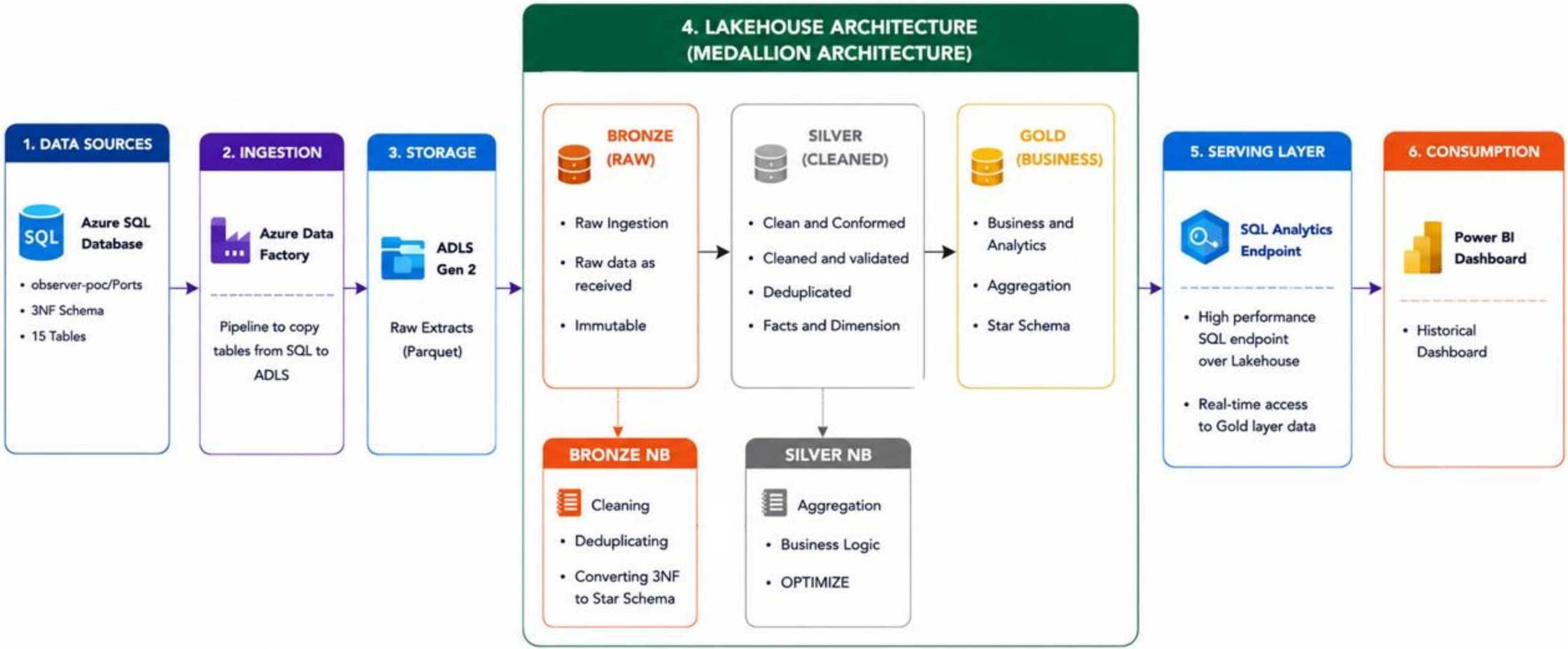
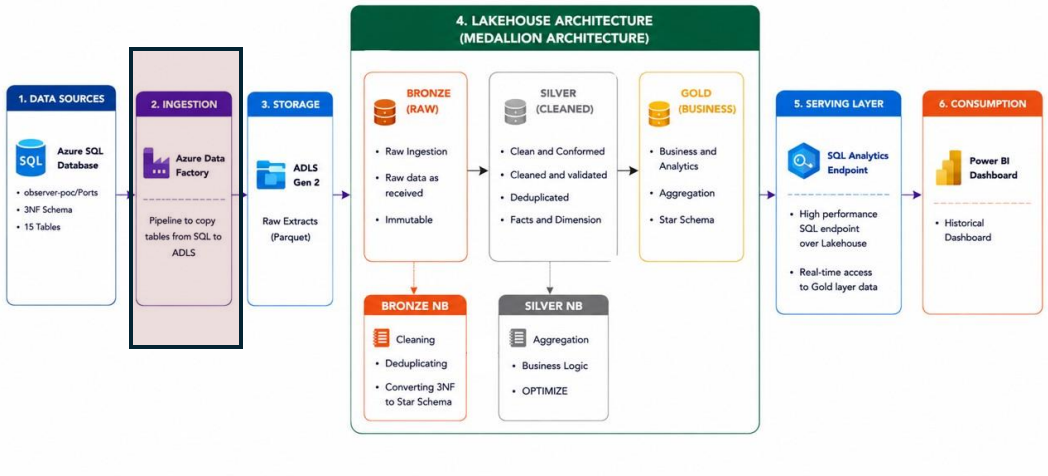
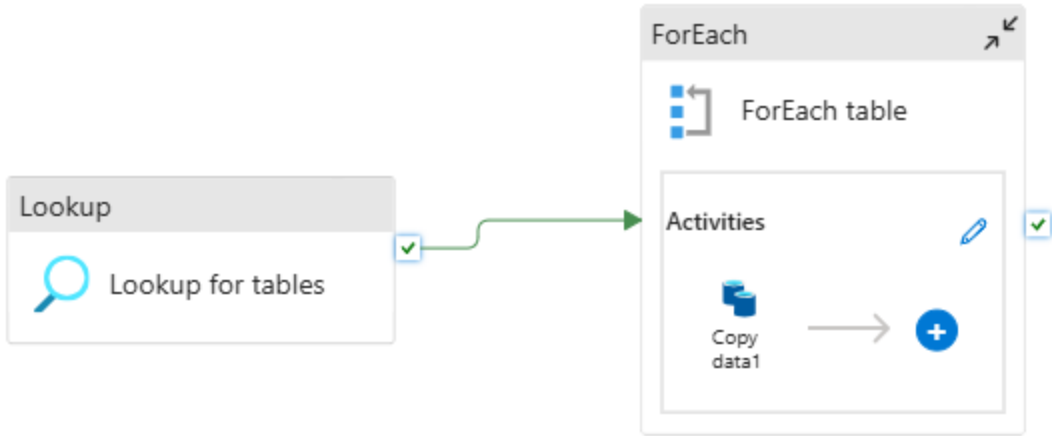
PortOps 360



An enterprise-scale end-to-end data engineering project built on Azure and Microsoft Fabric that combines batch processing and real-time streaming to provide historical business intelligence and live operational visibility for modern container ports.

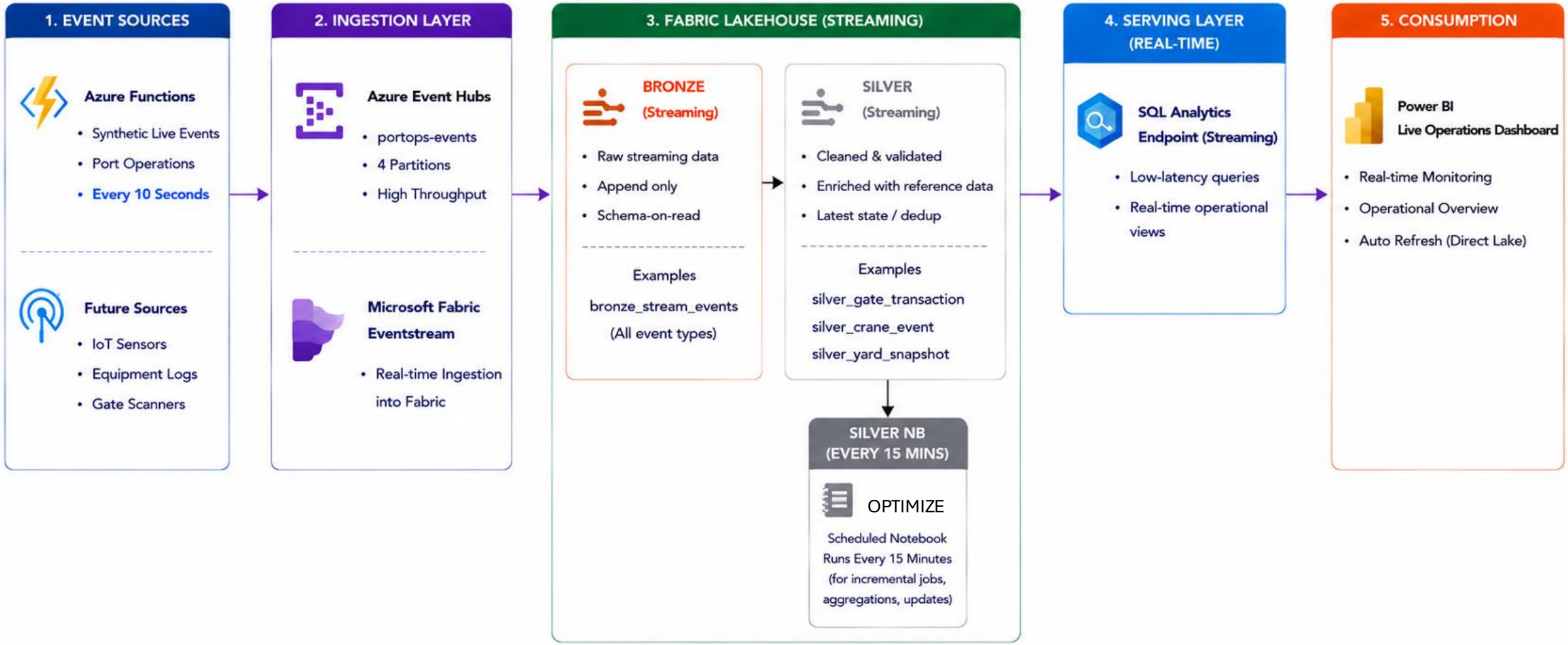

1. DATA SOURCES
Azure SQL Database
• observer-poc/Ports
• 3NF Schema
• 15 Tables
2. INGESTION
Azure Data Factory
Pipeline to copy tables from SQL to ADLS
3. STORAGE
ADLS Gen 2
Raw Extracts (Parquet)
4. LAKEHOUSE ARCHITECTURE (MEDALLION ARCHITECTURE)
BRONZE (RAW)
• Raw Ingestion
• Raw data as received
• Immutable
SILVER (CLEANED)
• Clean and Conformed
• Cleaned and validated
• Deduplicated
• Facts and Dimension
GOLD (BUSINESS)
• Business and Analytics
• Aggregation
• Star Schema
BRONZE NB
Cleaning
• Deduplicating
• Converting 3NF to Star Schema
SILVER NB
Aggregation
• Business Logic
• OPTIMIZE
5. SERVING LAYER
SQL Analytics Endpoint
• High performance SQL endpoint over Lakehouse
• Real-time access to Gold layer data
6. CONSUMPTION
Power BI Dashboard
• Historical Dashboard

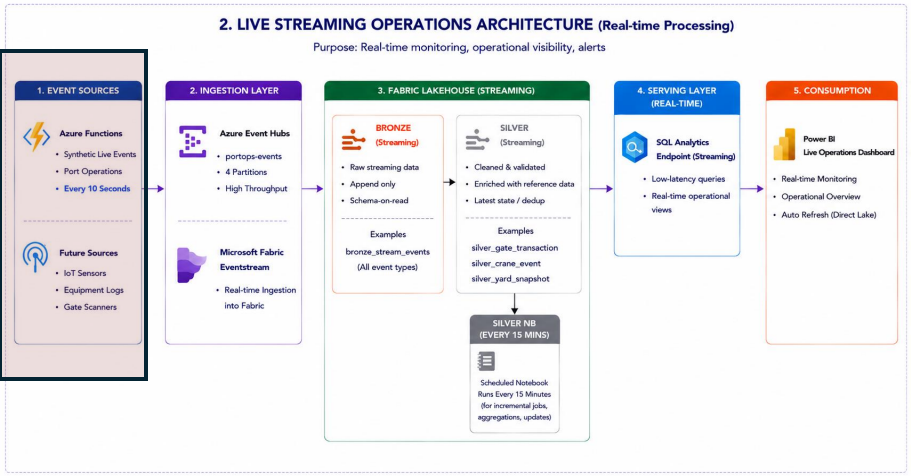

1. DATA SOURCES
Azure SQL Database
• observer-poc/Ports
• 3NF Schema
• 15 Tables
2. INGESTION
Azure Data Factory
Pipeline to copy tables from SQL to ADLS
3. STORAGE
ADLS Gen 2
Raw Extracts (Parquet)
4. LAKEHOUSE ARCHITECTURE (MEDALLION ARCHITECTURE)
BRONZE (RAW)
• Raw Ingestion
• Raw data as received
• Immutable
SILVER (CLEANED)
• Clean and Conformed
• Cleaned and validated
• Deduplicated
• Facts and Dimension
GOLD (BUSINESS)
• Business and Analytics
• Aggregation
• Star Schema
BRONZE NB
Cleaning
• Deduplicating
• Converting 3NF to Star Schema
SILVER NB
Aggregation
• Business Logic
• OPTIMIZE
5. SERVING LAYER
SQL Analytics Endpoint
• High performance SQL endpoint over Lakehouse
• Real-time access to Gold layer data
6. CONSUMPTION
Power BI Dashboard
• Historical Dashboard


Lookup
Lookup for tables
ForEach
ForEach table
Activities
Copy data1

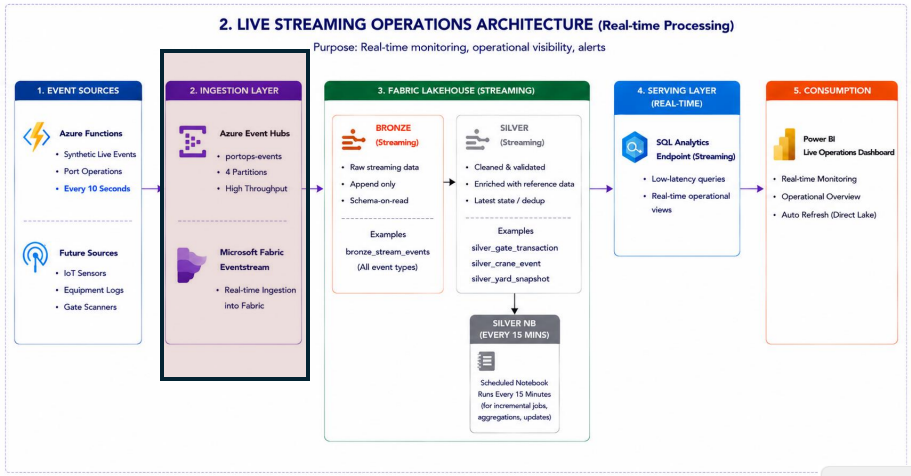
2. LIVE STREAMING OPERATIONS ARCHITECTURE (Real-time Processing)

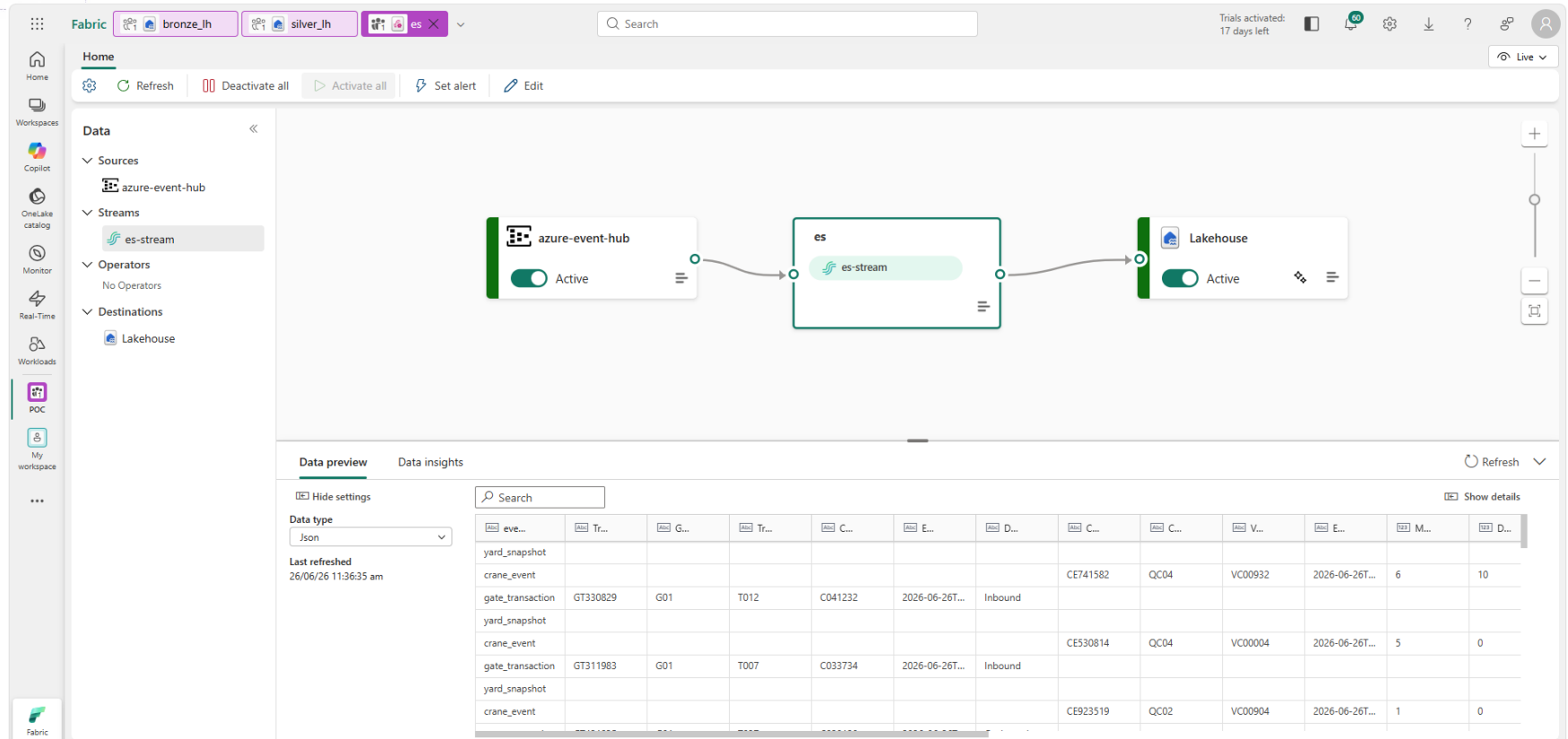
Purpose: Real-time monitoring, operational visibility, alerts

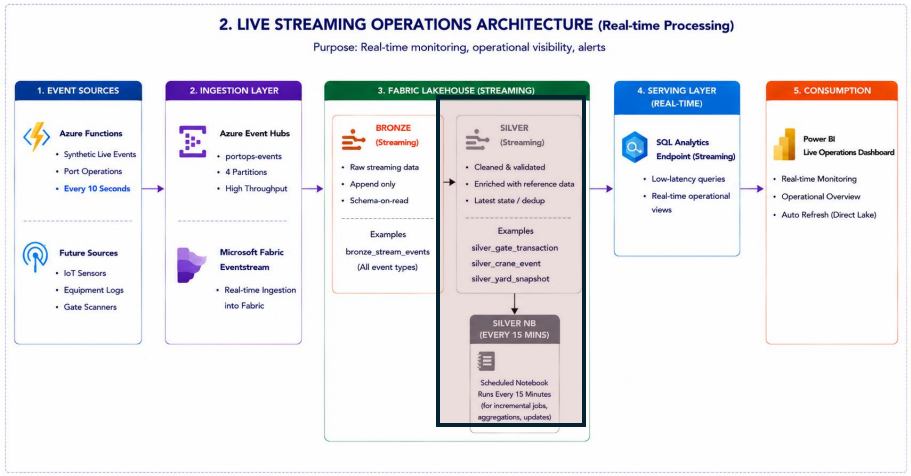





```python
@app.timer_trigger(
    schedule="*/10 * * * * *",
    arg_name="timer",
    run_on_startup=True
)
def stream_port_events(timer: func.TimerRequest):

    events = [
        generate_gate_transaction(),
        generate_crane_event(),
        generate_yard_snapshot()
    ]

    batch = producer.create_batch()

    for event in events:
        batch.add(EventData(json.dumps(event)))

    producer.send_batch(batch)

    print(
        f"{datetime.utcnow()} : Sent {len(events)} events"
    )
```


2. LIVE STREAMING OPERATIONS ARCHITECTURE (Real-time Processing)
Purpose: Real-time monitoring, operational visibility, alerts
1. EVENT SOURCES
Azure Functions
Synthetic Live Events
Port Operations
Every 10 Seconds
Future Sources
IoT Sensors
Equipment Logs
Gate Scanners
2. INGESTION LAYER
Azure Event Hubs
portops-events
4 Partitions
High Throughput
Microsoft Fabric Eventstream
Real-time Ingestion into Fabric
3. FABRIC LAKEHOUSE (STREAMING)
BRONZE (Streaming)
Raw streaming data
Append only
Schema-on-read
Examples
bronze_stream_events
(All event types)
SILVER (Streaming)
Cleaned & validated
Enriched with reference data
Latest state / dedup
Examples
silver_gate_transaction
silver_crane_event
silver_yard_snapshot
SILVER NB (EVERY 15 MINS)
Scheduled Notebook Runs Every 15 Minutes (for incremental jobs, aggregations, updates)
4. SERVING LAYER (REAL-TIME)
SQL Analytics Endpoint (Streaming)
Low-latency queries
Real-time operational views
5. CONSUMPTION
Power BI Live Operations Dashboard
Real-time Monitoring
Operational Overview
Auto Refresh (Direct Lake)


Fabric
bronze_lh
silver_lh
es
Search
Trials activated: 17 days left
Home
Refresh
Deactivate all
Activate all
Set alert
Edit
Live
Data
Sources
azure-event-hub
Streams
es-stream
Operators
No Operators
Destinations
Lakehouse
azure-event-hub
Active
es
es-stream
Lakehouse
Active
Data preview
Data insights
Refresh
Hide settings
Search
Show details
Data type
Json
Last refreshed
26/06/26 11:36:35 am
eve... | Tr... | G... | Tr... | C... | E... | D... | C... | C... | V... | E... | M... | D...
yard_snapshot
crane_event | CE741582 | QC04 | VC00932 | 2026-06-26T... | 6 | 10
gate_transaction | GT330829 | G01 | T012 | C041232 | 2026-06-26T... | Inbound
yard_snapshot
crane_event | CE530814 | QC04 | VC00004 | 2026-06-26T... | 5 | 0
gate_transaction | GT311983 | G01 | T007 | C033734 | 2026-06-26T... | Inbound
yard_snapshot
crane_event | CE923519 | QC02 | VC00904 | 2026-06-26T... | 1 | 0



Create Gate Transaction Silver

```
gate_df = df.filter(
    df.event_type == "gate_transaction"
)
```

StatementMeta(, 92c388d4-14af-4030-bd20-e54486a0fcdd, 12, Finish

```
from pyspark.sql.functions import col

gate_df = gate_df.select(
    "TransactionID",
    "GateID",
    "TruckID",
    "ContainerID",
    "EntryTime",
    "Direction",
    "CreatedUtc",
    "EventEnqueuedUtcTime"
)
```

```
from pyspark.sql.functions import to_timestamp

gate_df = gate_df \
    .withColumn(
        "EntryTime",
        to_timestamp("EntryTime")
    ) \
    .withColumn(
        "CreatedUtc",
        to_timestamp("CreatedUtc")
    )
```

StatementMeta(, 92c388d4-14af-4030-bd20-e54486a0fcdd, 14, Finish

```
dim_gate = spark.read.format("delta").load(
    "abfss://POC@onelake.dfs.fabric.microsoft.com/silver_lh.Lakehouse/Tables/dim_gate"
)
```

StatementMeta(, 92c388d4-14af-4030-bd20-e54486a0fcdd, 15, Finished, Available, Finished,

```
gate_df = gate_df.dropDuplicates(
    ["TransactionID"]
)

gate_df = gate_df.join(
    dim_gate,
    "GateID",
    "left"
)
```

StatementMeta(, 92c388d4-14af-4030-bd20-e54486a0fcdd, 16, Finished, Available, Finished,

```
gate_df.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("silver_gate_transaction")
```

Dashboards

LMS-Submission-Aman-Rajput/POC/Port POC/Screenshots/Dashboard at master · AmanRajputDE/LMS-Submission-Aman-Rajput

TECHNOLOGY

Built on enterprise-grade Azure and Fabric

A modern cloud-native stack from ingestion to visualization, spanning batch and streaming workloads.

AZURE SERVICES

- Azure SQL Database
- Azure Data Factory
- ADLS Gen 2
- Azure Functions
- Azure Event Hubs

MICROSOFT FABRIC

- Lakehouse
- Eventstream
- Pipelines
- Notebooks
- SQL Endpoint
- Semantic Models
- Power BI
- Direct Lake

DATA ENGINEERING PATTERNS

- Medallion Architecture
- Delta Lake
- Star Schema
- 3NF Database Design
- Event-Driven Architecture
- Real-Time Streaming
- Lakehouse Architecture
- Dimensional Modeling

LANGUAGES

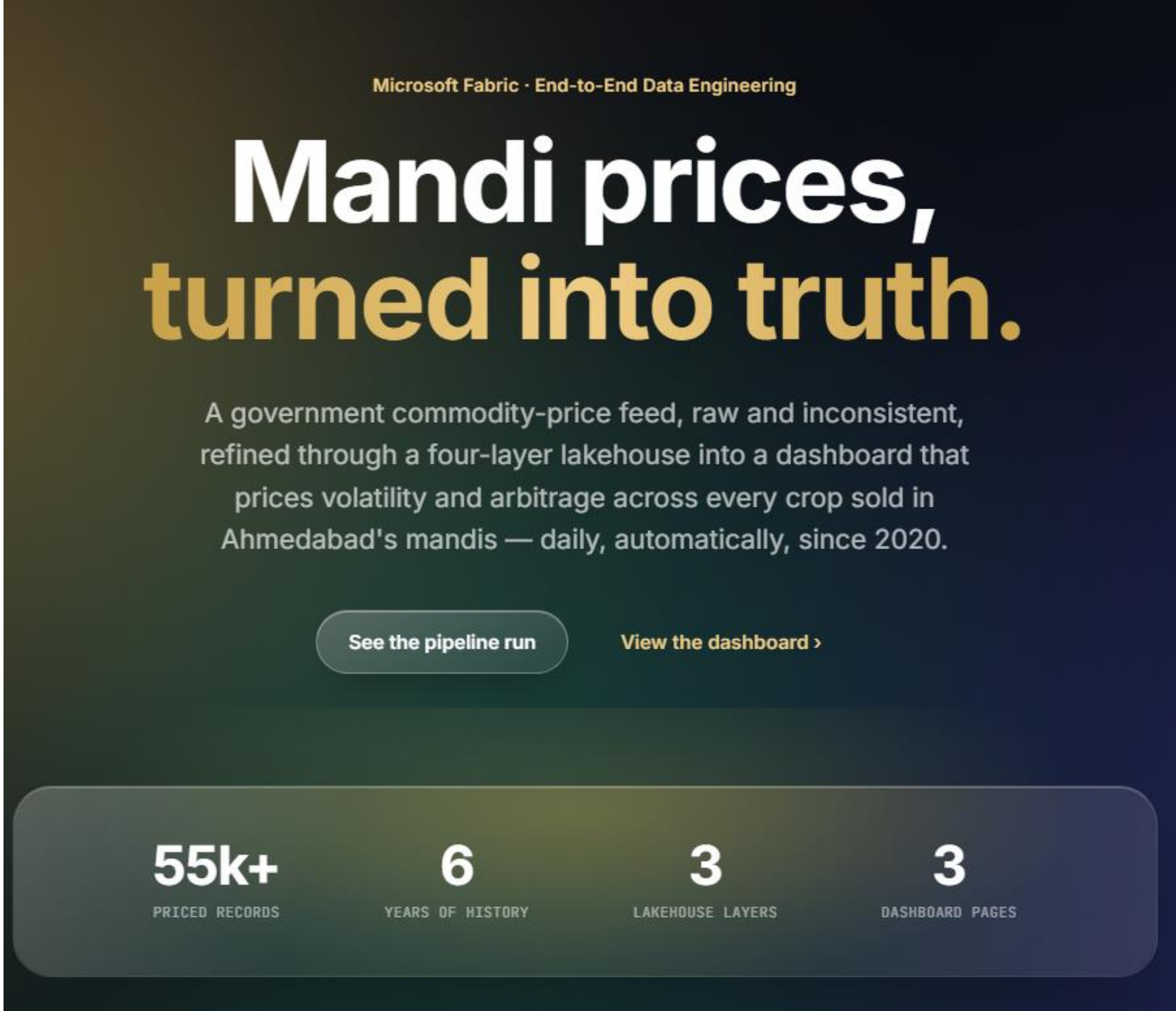
- Python
- PySpark
- SQL
- DAX

India Mandi Price Intelligence Platform

India has **7,000+** regulated wholesale markets (mandis) trading **300+** commodities daily.

- A farmer in Ahmedabad has no way to know if a mandi 50 km away is offering 30% more for their produce today

- Traders exploit this information gap, pocketing the price difference as margin



01

One district. One API. Every single day.

The platform is deliberately narrow: **Ahmedabad district, Gujarat**, sourced from the data.gov.in Mandi commodity-price resource. Scope was chosen over coverage — a smaller surface meant the pipeline could be made genuinely correct, end to end, rather than broad and shallow.

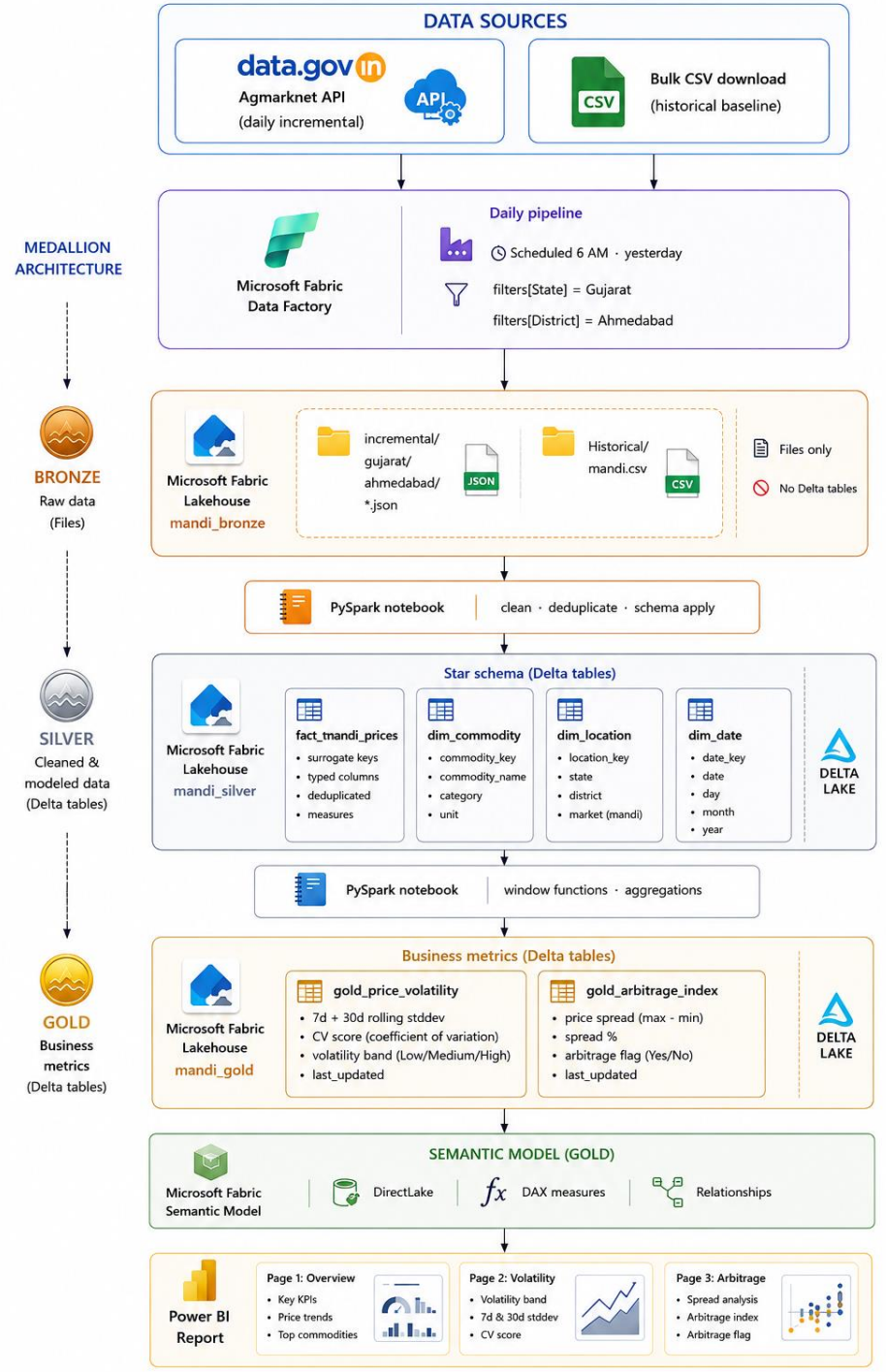
Two streams feed the lakehouse: a one-time **historical CSV** backfilled to 2020, and a **daily incremental pull** that has run automatically ever since, one calendar day behind real time.

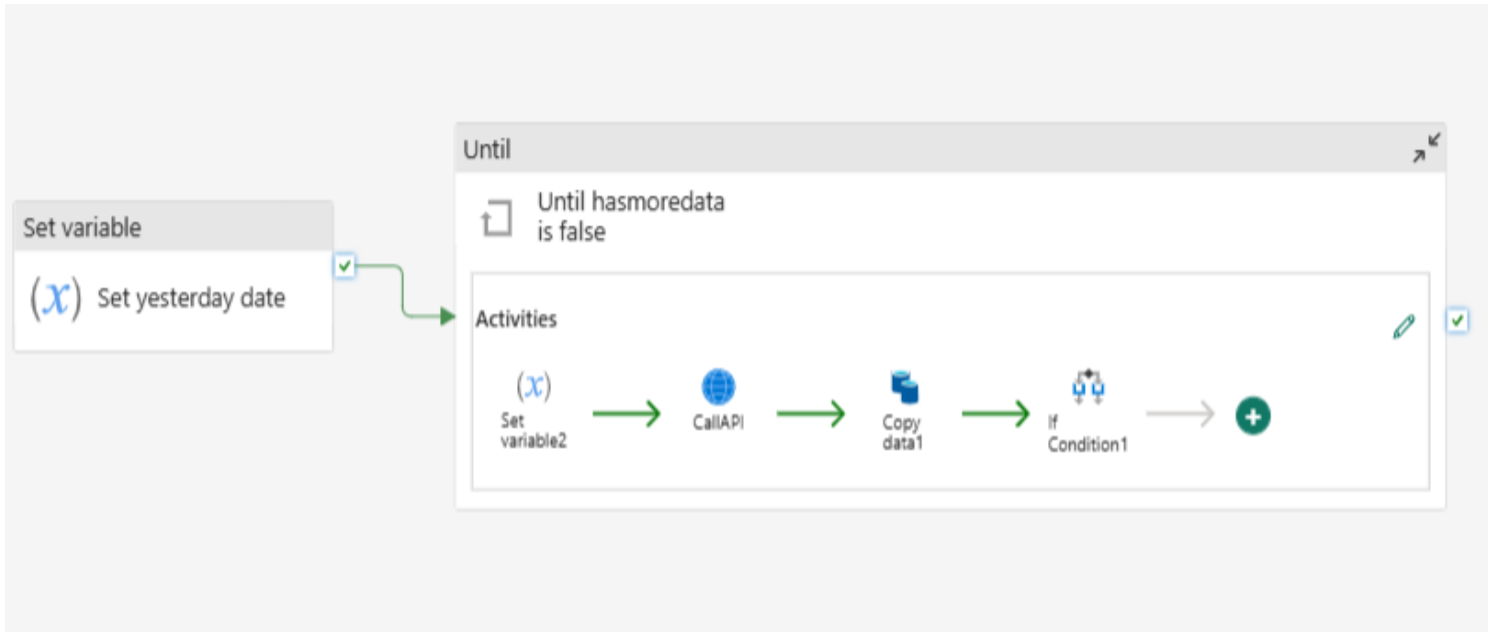
REST ENDPOINT

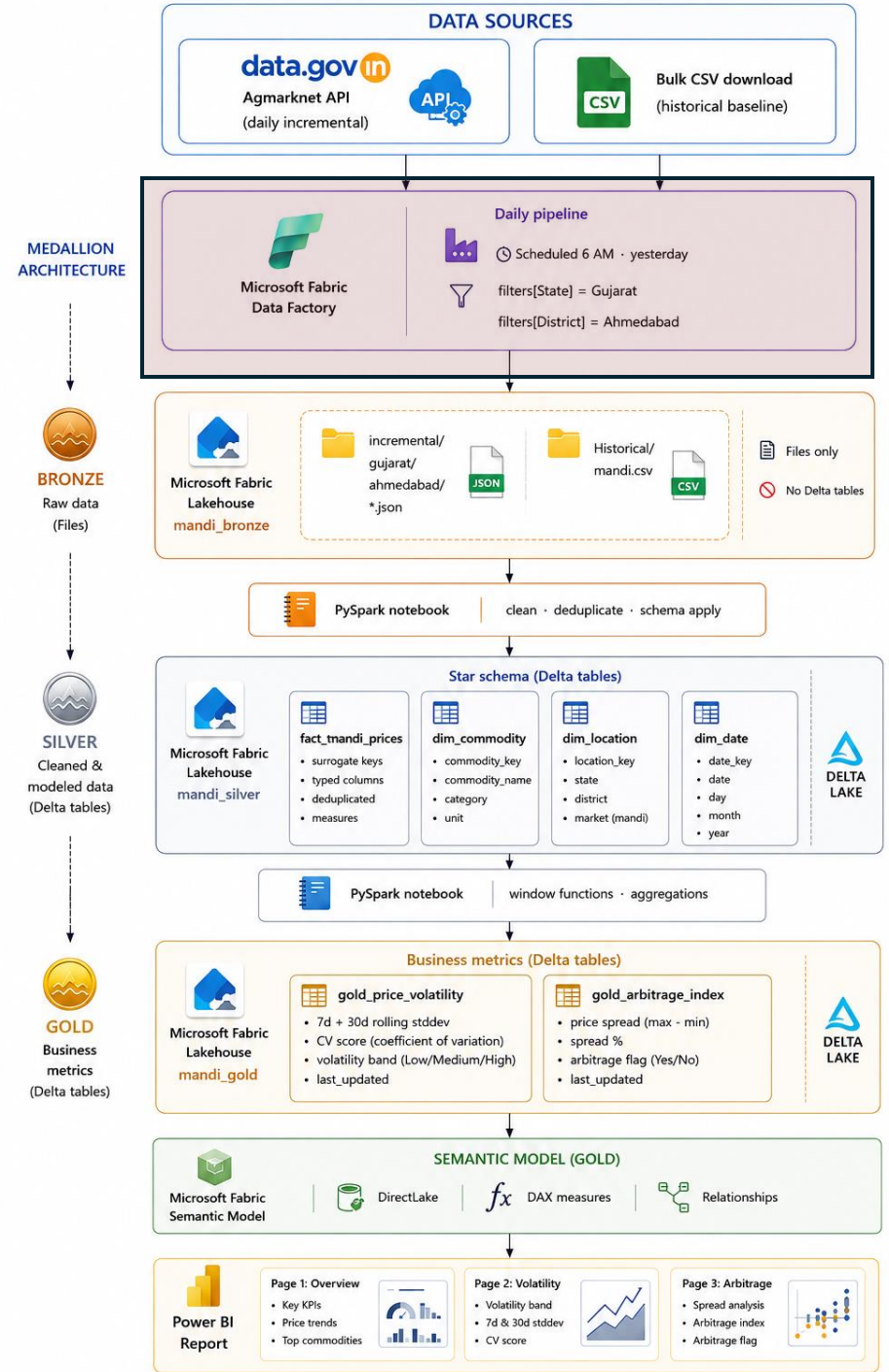
```
GET /resource/35985678-0d79-46b4-9ed6-6f13308a1d24
  ?api-key=••••••••••••••••
  &filters[State]=Gujarat
  &filters[District]=Ahmedabad
  &filters[Arrival_Date]={targetDate}
  &offset={offset}&limit=100&format=json
```

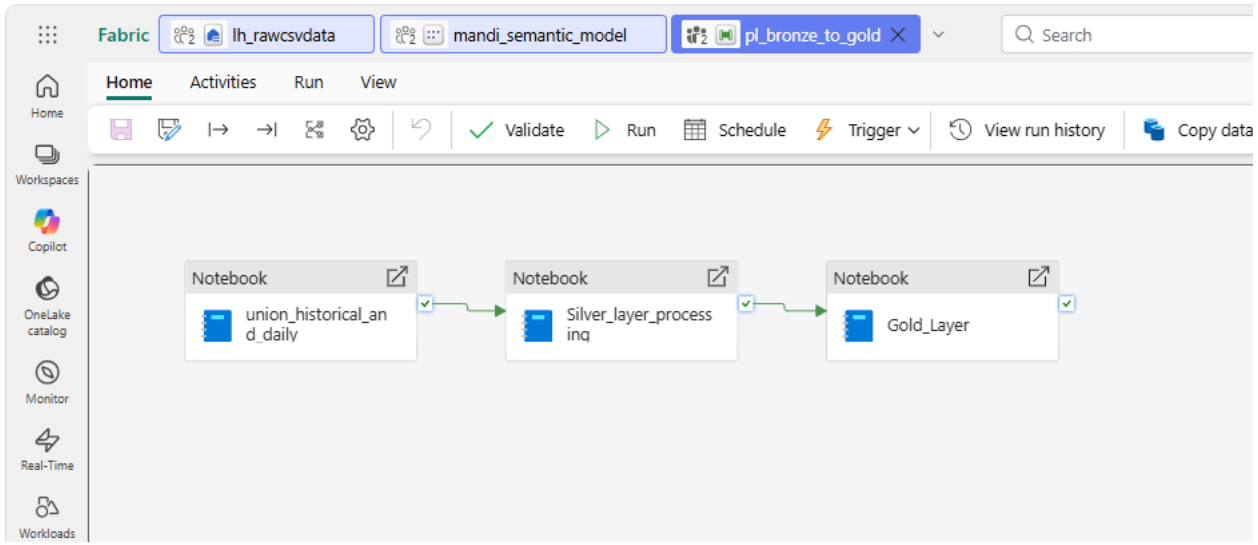
Filtered server-side — only Ahmedabad ever reaches the lakehouse.

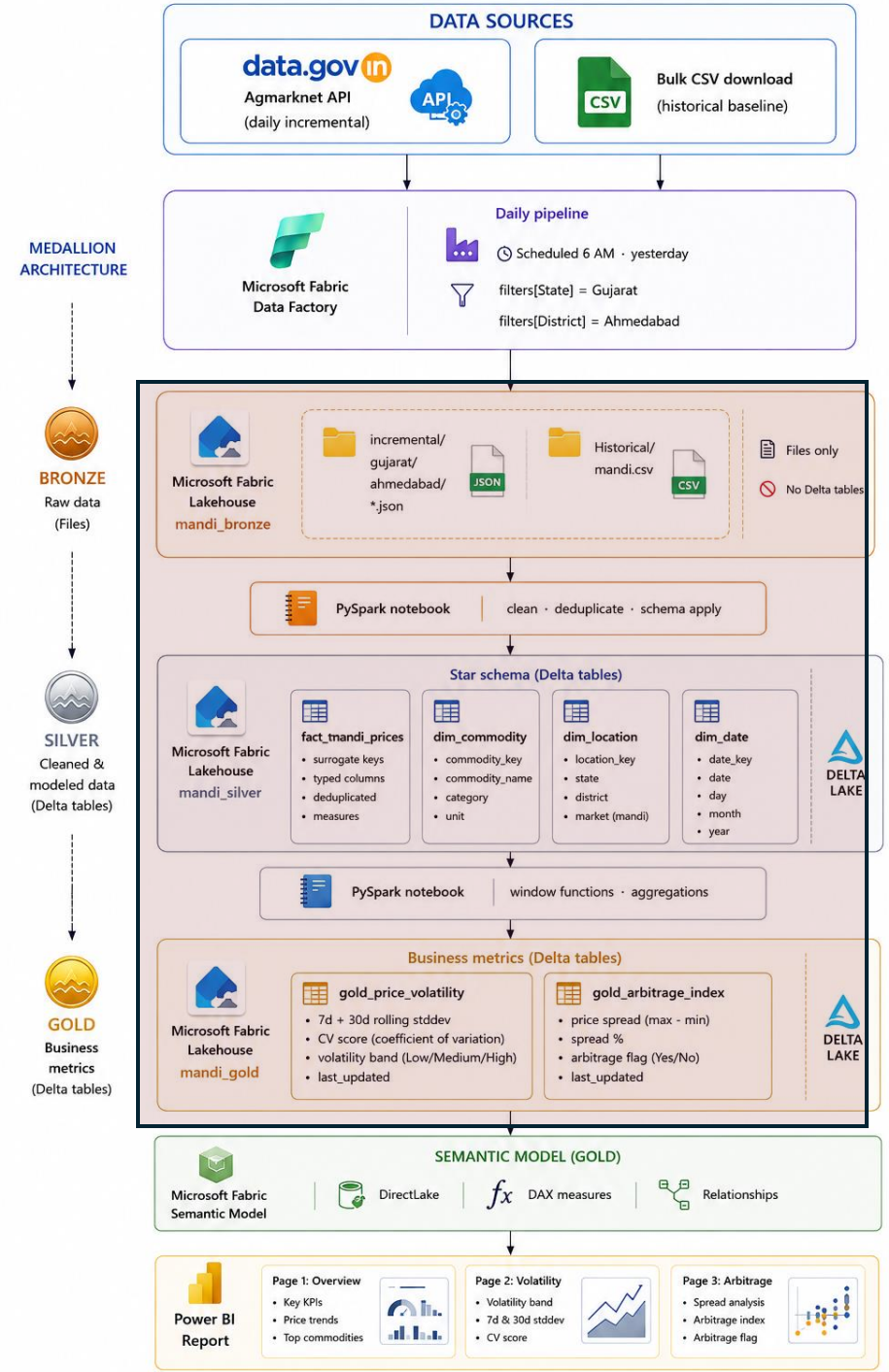
Architecture

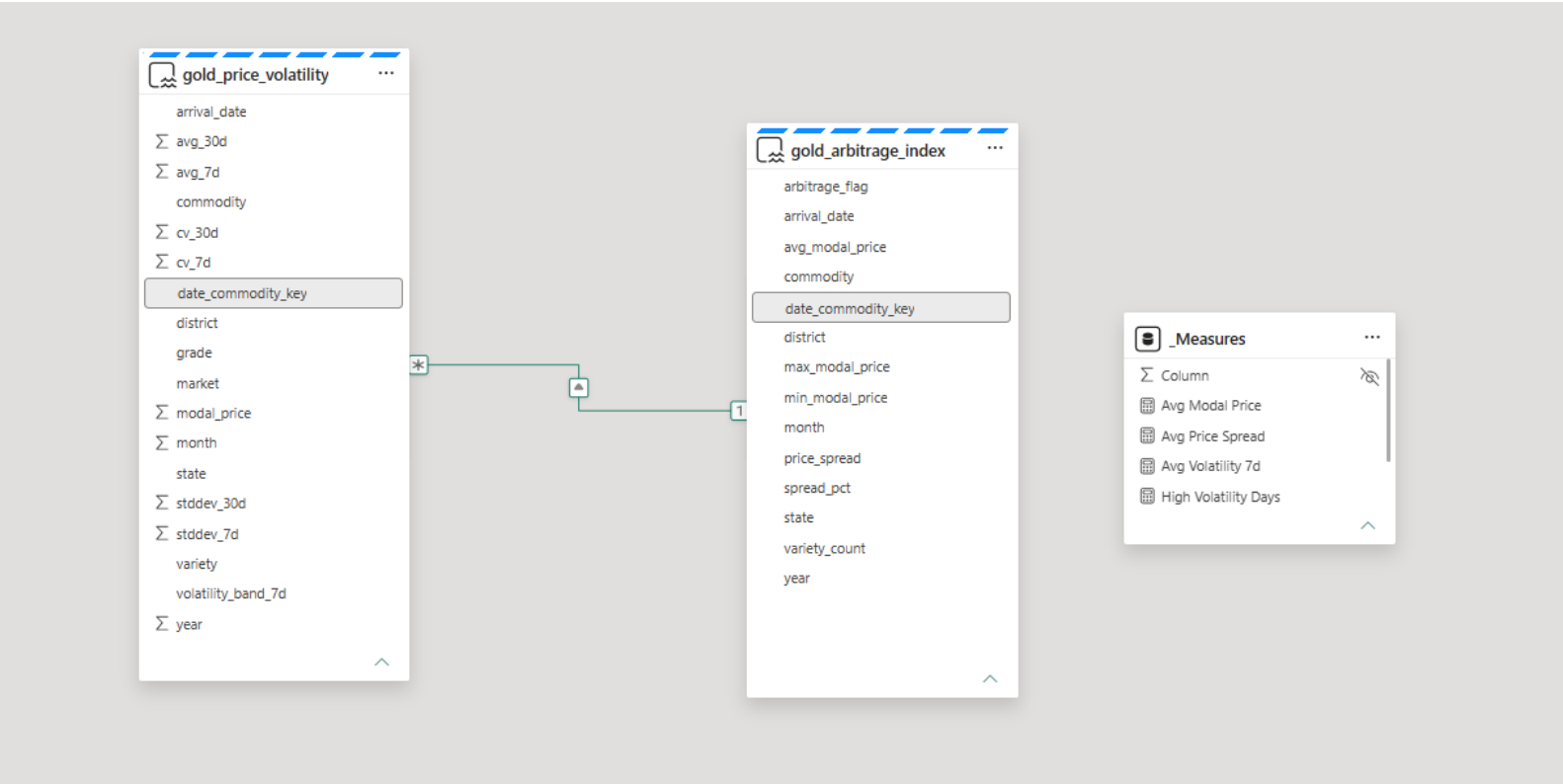


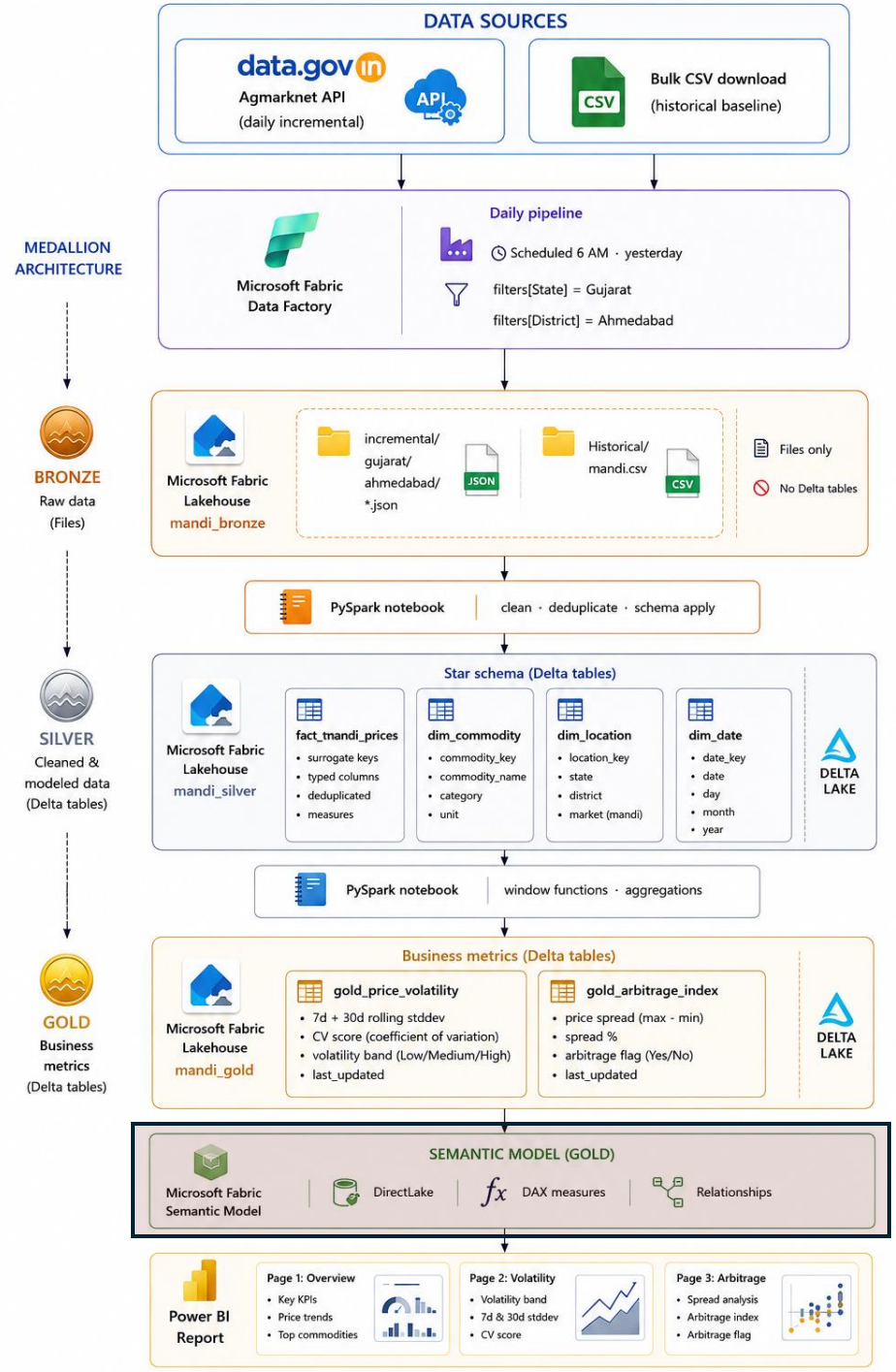

Set variable
Set yesterday date
Until
Until hasmoredata is false
Activities
Set variable2
CallAPI
Copy data1
If Condition1


DATA SOURCES
data.gov.in
Agmarknet API
(daily incremental)
API
CSV
Bulk CSV download
(historical baseline)
MEDALLION ARCHITECTURE
Microsoft Fabric Data Factory
Daily pipeline
Scheduled 6 AM · yesterday
filters[State] = Gujarat
filters[District] = Ahmedabad
BRONZE
Raw data (Files)
Microsoft Fabric Lakehouse
mandi_bronze
incremental/ gujarat/ ahmedabad/ *.json
JSON
Historical/ mandi.csv
CSV
Files only
No Delta tables
PySpark notebook
clean · deduplicate · schema apply
SILVER
Cleaned & modeled data (Delta tables)
Star schema (Delta tables)
Microsoft Fabric Lakehouse
mandi_silver
fact_tmandi_prices
surrogate keys
typed columns
deduplicated
measures
dim_commodity
commodity_key
commodity_name
category
unit
dim_location
location_key
state
district
market (mandi)
dim_date
date_key
date
day
month
year
DELTA LAKE
PySpark notebook
window functions · aggregations
GOLD
Business metrics (Delta tables)
Business metrics (Delta tables)
Microsoft Fabric Lakehouse
mandi_gold
gold_price_volatility
7d + 30d rolling stddev
CV score (coefficient of variation)
volatility band (Low/Medium/High)
last_updated
gold_arbitrage_index
price spread (max - min)
spread %
arbitrage flag (Yes/No)
last_updated
DELTA LAKE
SEMANTIC MODEL (GOLD)
Microsoft Fabric Semantic Model
DirectLake
DAX measures
Relationships
Power BI Report
Page 1: Overview
Key KPIs
Price trends
Top commodities
Page 2: Volatility
Volatility band
7d & 30d stddev
CV score
Page 3: Arbitrage
Spread analysis
Arbitrage index
Arbitrage flag


Fabric
lh_rawcsvdata
mandi_semantic_model
pl_bronze_to_gold
Search
Home
Activities
Run
View
Validate
Run
Schedule
Trigger
View run history
Copy data
Workspaces
Copilot
OneLake catalog
Monitor
Real-Time
Workloads
Notebook
union_historical_and_daily
Notebook
Silver_layer_processing
Notebook
Gold_Layer


DATA SOURCES
data.gov.in
Agmarknet API
(daily incremental)
API
CSV
Bulk CSV download
(historical baseline)
MEDALLION ARCHITECTURE
Microsoft Fabric Data Factory
Daily pipeline
Scheduled 6 AM · yesterday
filters[State] = Gujarat
filters[District] = Ahmedabad
BRONZE
Raw data (Files)
Microsoft Fabric Lakehouse
mandi_bronze
incremental/ gujarat/ ahmedabad/ *.json
JSON
Historical/ mandi.csv
CSV
Files only
No Delta tables
PySpark notebook
clean · deduplicate · schema apply
SILVER
Cleaned & modeled data (Delta tables)
Star schema (Delta tables)
Microsoft Fabric Lakehouse
mandi_silver
fact_mandi_prices
surrogate keys
typed columns
deduplicated
measures
dim_commodity
commodity_key
commodity_name
category
unit
dim_location
location_key
state
district
market (mandi)
dim_date
date_key
date
day
month
year
DELTA LAKE
PySpark notebook
window functions · aggregations
GOLD
Business metrics (Delta tables)
Business metrics (Delta tables)
Microsoft Fabric Lakehouse
mandi_gold
gold_price_volatility
7d + 30d rolling stddev
CV score (coefficient of variation)
volatility band (Low/Medium/High)
last_updated
gold_arbitrage_index
price spread (max - min)
spread %
arbitrage flag (Yes/No)
last_updated
DELTA LAKE
SEMANTIC MODEL (GOLD)
Microsoft Fabric Semantic Model
DirectLake
DAX measures
Relationships
Power BI Report
Page 1: Overview
Key KPIs
Price trends
Top commodities
Page 2: Volatility
Volatility band
7d & 30d stddev
CV score
Page 3: Arbitrage
Spread analysis
Arbitrage index
Arbitrage flag


gold_price_volatility
arrival_date
avg_30d
avg_7d
commodity
cv_30d
cv_7d
date_commodity_key
district
grade
market
modal_price
month
state
stddev_30d
stddev_7d
variety
volatility_band_7d
year
gold_arbitrage_index
arbitrage_flag
arrival_date
avg_modal_price
commodity
date_commodity_key
district
max_modal_price
min_modal_price
month
price_spread
spread_pct
state
variety_count
year
_Measures
Column
Avg Modal Price
Avg Price Spread
Avg Volatility 7d
High Volatility Days


DATA SOURCES
data.gov.in
Agmarknet API
(daily incremental)
API
CSV
Bulk CSV download
(historical baseline)
MEDALLION ARCHITECTURE
Microsoft Fabric Data Factory
Daily pipeline
Scheduled 6 AM · yesterday
filters[State] = Gujarat
filters[District] = Ahmedabad
BRONZE
Raw data (Files)
Microsoft Fabric Lakehouse
mandi_bronze
incremental/ gujarat/ ahmedabad/ *.json
JSON
Historical/ mandi.csv
CSV
Files only
No Delta tables
PySpark notebook
clean · deduplicate · schema apply
SILVER
Cleaned & modeled data (Delta tables)
Star schema (Delta tables)
Microsoft Fabric Lakehouse
mandi_silver
fact_tmandi_prices
surrogate keys
typed columns
deduplicated
measures
dim_commodity
commodity_key
commodity_name
category
unit
dim_location
location_key
state
district
market (mandi)
dim_date
date_key
date
day
month
year
DELTA LAKE
PySpark notebook
window functions · aggregations
GOLD
Business metrics (Delta tables)
Business metrics (Delta tables)
Microsoft Fabric Lakehouse
mandi_gold
gold_price_volatility
7d + 30d rolling stddev
CV score (coefficient of variation)
volatility band (Low/Medium/High)
last_updated
gold_arbitrage_index
price spread (max - min)
spread %
arbitrage flag (Yes/No)
last_updated
DELTA LAKE
SEMANTIC MODEL (GOLD)
Microsoft Fabric Semantic Model
DirectLake
DAX measures
Relationships
Power BI Report
Page 1: Overview
Key KPIs
Price trends
Top commodities
Page 2: Volatility
Volatility band
7d & 30d stddev
CV score
Page 3: Arbitrage
Spread analysis
Arbitrage index
Arbitrage flag

Data Factory

ORCHESTRATION

Two pipelines — `pl_mandi_daily_incremental` for ingestion, `pl_bronze_to_gold` for transformation. Web, Copy, Until and If-Condition activities, no custom scheduler.

Lakehouse

STORAGE · BRONZE & SILVER

`lh_rawcsvdata` holds the historical CSV and daily JSON batches. `lh_silver` holds the cleaned table and star schema.

OneLake

UNIFIED STORAGE LAYER

Every lakehouse in this workspace sits on the same OneLake account — one `abfss://onelake.dfs.fabric.microsoft.com` namespace, no separate storage account to provision.

Notebooks · Spark

TRANSFORMATION ENGINE

Three PySpark notebooks — `nb_union_hist_daily`, `nb_silver`, `nb_gold` — run on Fabric's managed Spark pool, chained as `TridentNotebook` activities.

Delta Tables

TABLE FORMAT

Every silver and gold output is written as managed Delta — `fact_mandi_prices`, the three dimension tables, and both gold marts — versioned and queryable straight from Power BI.

Power BI

SEMANTIC MODEL & REPORT

A semantic model joining both gold tables on `date_commodity_key`, feeding a three-page report — overview, arbitrage, and volatility.

Dashboards

[LMS-Submission-Aman-Rajput/POC/Mandi POC/Screenshot/Dashboard at master · AmanRajputDE/LMS-Submission-Aman-Rajput]